

C Programming Basics
Lab Report and Knowledge Check

Pravina kumari Chaudhary
Bsc (Hons) Computer Science with Artificial Intelligence
Softwarica College of IT and E-commerce
ST5003CMD Operating Systems, Security, and Networks

Rupak Rajbanshi
26th September, 2026

TABLE OF CONTENT

I. Introduction	4
II. Task 1: The Long-Running Process	5
A. C Source Code	5
B. Terminal Execution & Verification	6
III. Task 2: Process Identity (PID and PPID)	7
A. C Source Code	7
B. Terminal Execution & Verification	8
IV. Task 3: Exit Codes and OS Feedback	9
A. C Source Code	9
B. Terminal Execution & Verification	10
V. Task 4: Standard I/O Streams	11
A. C Source Code	11
B. Terminal Execution & Verification	12
VI. Task 5: Conditional Execution and Termination	13
A. C Source Code	13
B. Terminal Execution & Verification	14
VII. Conclusion	16
GITHUB REPOSITORY.....	16
Appendix.....	17

LIST OF FIGURES

Figure 1. Source Code (Task 1)	5
Figure 2. Output (Task 1)	6
Figure 3. Flow diagram of task1_alive.c	6
Figure 4. Source Code (Task 2)	7
Figure 5. Output (Task 2)	8
Figure 6. Flow diagram of task2_identity.c	8
Figure 7. Source Code (Task 3)	9
Figure 8. Output (Task 3).....	10
Figure 9. Flow diagram of task3_exit.c	10
Figure 10. Source Code (Task 4)	11
Figure 11. Output (Task 4)	12
Figure 12. Flow diagram of task4_input.c	12
Figure 13. Source Code (Task 5)	13
Figure 14. Output (Task 5)	14
Figure 15. Flow diagram of task5_control.c	15

I. Introduction

In Lecture 3, we studied how an operating system handles processes and resource allocation. This lab puts that theory into practice. Rather than just writing code that prints output, we write code that communicates directly with the Linux kernel — querying its own identity, controlling how long it runs, and reporting back its outcome to the shell.

All five programs were written in C using the nano editor, compiled using GCC, and run on Ubuntu 24.04 under WSL. Each task includes the source code, a flow diagram, the terminal commands used to verify it, and screenshots of the output.

II. Task1: The Long-Running Process

OS Insight: In enterprise servers, processes run for days or months. We must know how to monitor them while they are active.

Theory: A process is simply a program that is currently executing. As long as it runs, the kernel tracks it inside the process table, which can be viewed using commands like `ps`. Calling `sleep()` pauses the process for a set number of seconds — during this pause it sits in the sleeping (S) state and barely touches the CPU. Appending `&` to a command sends the process into the background, leaving the shell free to accept other commands.

A. C Source Code

```
GNU nano 8.7.1 task1_alive.c
#include <stdio.h>
#include <unistd.h>

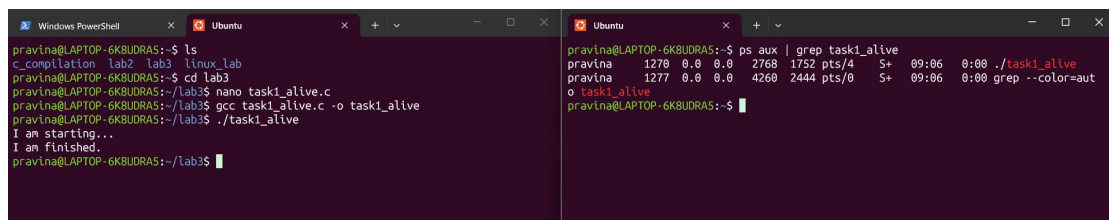
int main() {
    printf("I am starting...\n");

    // Loop for 30 seconds
    for(int i = 1; i <= 30; i++) {
        sleep(1); // Pauses execution for 1 second
    }

    printf("I am finished.\n");
    return 0;
}
```

Figure 1 : Source Code

B. Terminal Execution and Verification



The image shows two terminal windows. The left window, titled 'Windows PowerShell' and 'Ubuntu', shows the execution of the program. The right window, titled 'Ubuntu', shows the output of the 'ps aux | grep task1_alive' command.

Left Terminal (Windows PowerShell / Ubuntu):

```
pravin@LAPTOP-GK8UDRAS:~$ ls
C:\compilation lab2 lab3 linux lab
pravin@LAPTOP-GK8UDRAS:~$ cd lab3
pravin@LAPTOP-GK8UDRAS:~/lab3$ nano task1_alive.c
pravin@LAPTOP-GK8UDRAS:~/lab3$ gcc task1_alive.c -o task1_alive
pravin@LAPTOP-GK8UDRAS:~/lab3$ ./task1_alive
I am starting...
I am finished.
pravin@LAPTOP-GK8UDRAS:~/lab3$
```

Right Terminal (Ubuntu):

```
pravin@LAPTOP-GK8UDRAS:~$ ps aux | grep task1_alive
pravin   1270  0.0  0.0   2768 1752 pts/4    S+   09:06   0:00 ./task1_alive
pravin   1277  0.0  0.0   4260 2444 pts/0    S+   09:06   0:00 grep --color=auto task1_alive
pravin@LAPTOP-GK8UDRAS:~$
```

Figure 2 : Output

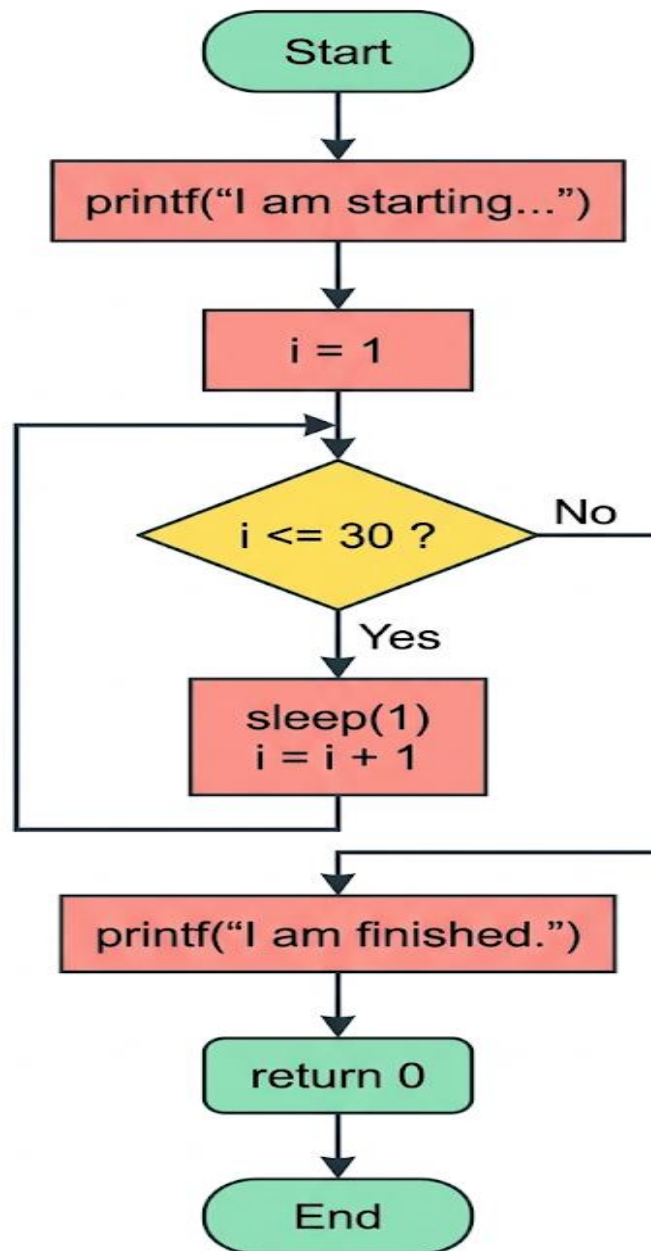


Figure 3 : Flow diagram of `task1_alive.c`

III. Task 2: Process Identity (PID and PPID)

OS Insight: Every process in Linux has a unique Process ID (PID). The process that created it is the Parent Process ID (PPID), usually your terminal shell.

Theory: Every process in Linux is identified by a unique Process ID (PID), assigned by the kernel when the process is created. The process that started it — usually the shell — is called its parent, and its PID is called the Parent Process ID (PPID). In C, the `getpid()` function returns the calling process's own PID, and `getppid()` returns its parent's PID. This parent-child relationship forms the basis of how Linux organizes and tracks all running processes in the process table.

A. C Source Code



```
GNU nano 8.7.1 task2_identity.c
#include <stdio.h>
#include <unistd.h>

int main() {
    // Get current Process ID
    pid_t my_pid = getpid();

    // Get Parent Process ID
    pid_t my_ppid = getppid();

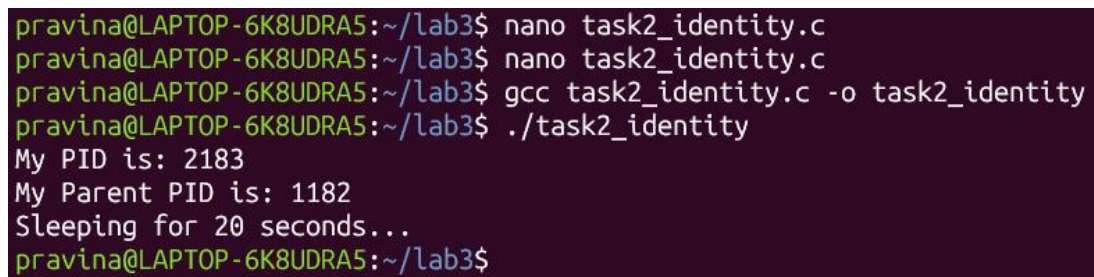
    printf("My PID is: %d\n", my_pid);
    printf("My Parent PID is: %d\n", my_ppid);

    printf("Sleeping for 20 seconds...\n");
    sleep(20);

    return 0;
}
```

Figure 4 : Source Code

B. Terminal Execution and Verification

A terminal window with a dark purple background and light green text. The prompt is 'pravina@LAPTOP-6K8UDRA5:~/lab3\$'. The user enters 'nano task2_identity.c', which is repeated. Then 'gcc task2_identity.c -o task2_identity' is entered. Finally, './task2_identity' is entered, resulting in the output: 'My PID is: 2183', 'My Parent PID is: 1182', and 'Sleeping for 20 seconds...'. The prompt returns to 'pravina@LAPTOP-6K8UDRA5:~/lab3\$'.

```
pravina@LAPTOP-6K8UDRA5:~/lab3$ nano task2_identity.c
pravina@LAPTOP-6K8UDRA5:~/lab3$ nano task2_identity.c
pravina@LAPTOP-6K8UDRA5:~/lab3$ gcc task2_identity.c -o task2_identity
pravina@LAPTOP-6K8UDRA5:~/lab3$ ./task2_identity
My PID is: 2183
My Parent PID is: 1182
Sleeping for 20 seconds...
pravina@LAPTOP-6K8UDRA5:~/lab3$
```

Figure 5 : Output

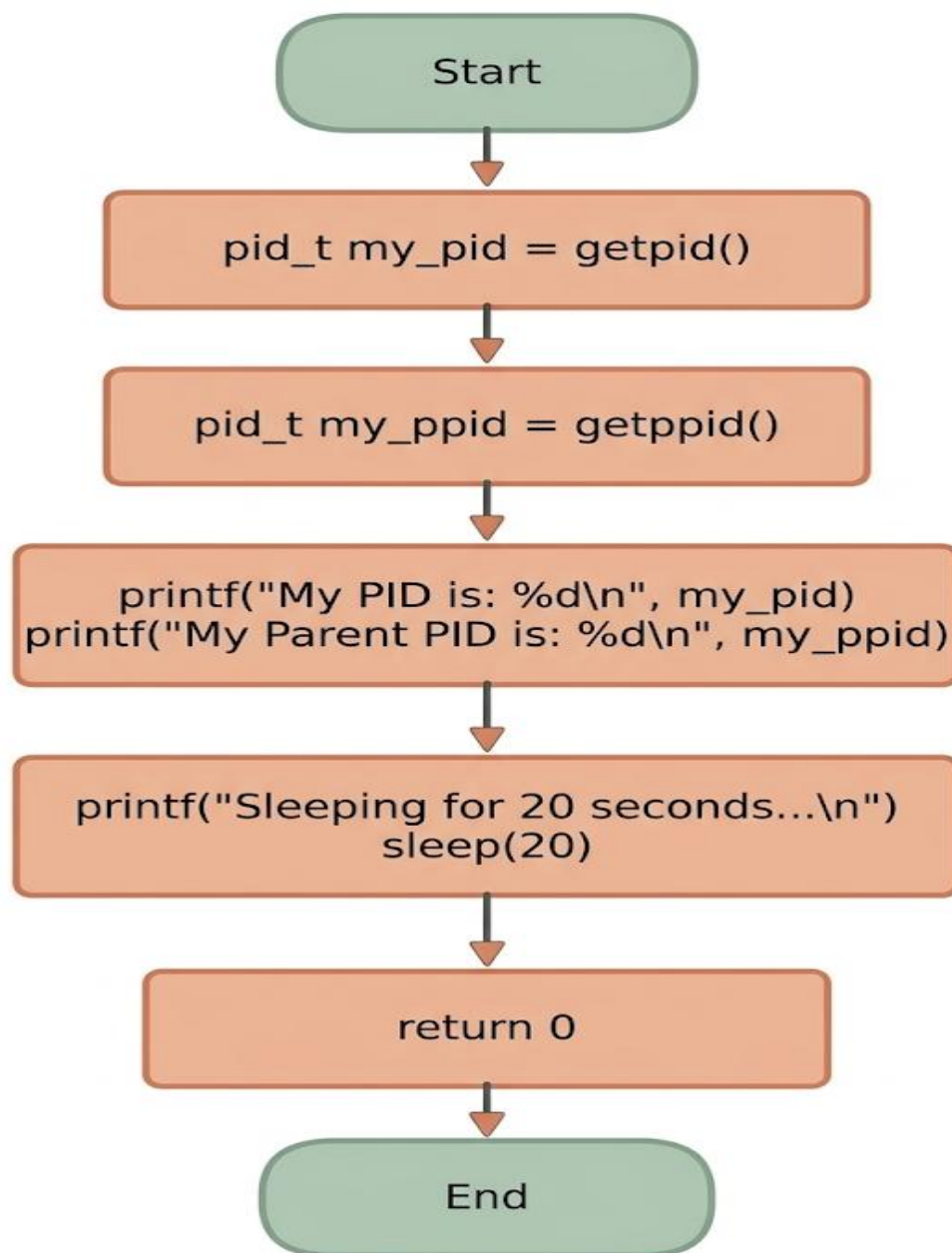


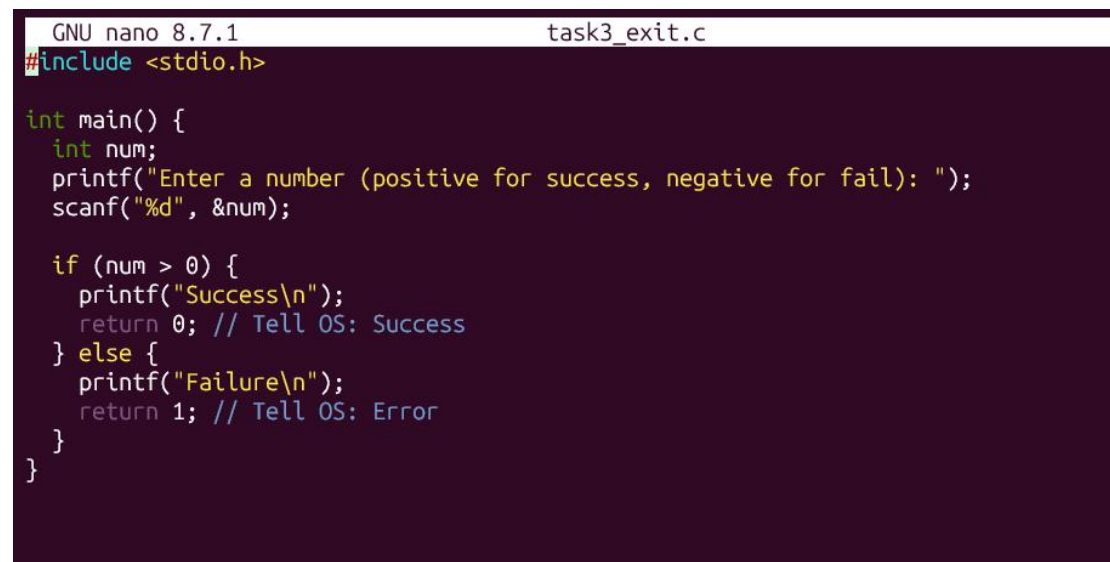
Figure 6 : Flow diagram of task2_identity.c

IV. Task 3: Exit Codes and OS Feedback

OS Insight: Programs communicate success or failure to the OS using exit codes. A return value of 0 means success; a non-zero value means an error.

Theory: When a program finishes execution, it returns an integer value to the operating system called an exit code (or exit status). By convention, an exit code of 0 means the program completed successfully, while any non-zero value indicates some kind of error or failure. In C, this value is set using return inside main(), and the shell stores the exit code of the last executed command in the special variable \$?, which can be checked immediately after running the program.

A. C Source Code



```
GNU nano 8.7.1 task3_exit.c
#include <stdio.h>

int main() {
    int num;
    printf("Enter a number (positive for success, negative for fail): ");
    scanf("%d", &num);

    if (num > 0) {
        printf("Success\n");
        return 0; // Tell OS: Success
    } else {
        printf("Failure\n");
        return 1; // Tell OS: Error
    }
}
```

Figure 7 : Source Code

B. Terminal Execution and Verification

```
pravina@LAPTOP-6K8UDRA5:~/lab3$ nano task3_exit.c
pravina@LAPTOP-6K8UDRA5:~/lab3$ nano task3_exit.c
pravina@LAPTOP-6K8UDRA5:~/lab3$ gcc task3_exit.c -o task3
pravina@LAPTOP-6K8UDRA5:~/lab3$ ./task3_exit
-bash: ./task3_exit: No such file or directory
pravina@LAPTOP-6K8UDRA5:~/lab3$ ./task3
Enter a number (positive for success, negative for fail): 5
Success
pravina@LAPTOP-6K8UDRA5:~/lab3$ echo $?
0
pravina@LAPTOP-6K8UDRA5:~/lab3$ ./task3
Enter a number (positive for success, negative for fail): -5
Failure
pravina@LAPTOP-6K8UDRA5:~/lab3$ echo $?
1
pravina@LAPTOP-6K8UDRA5:~/lab3$
```

Figure 8 : Output

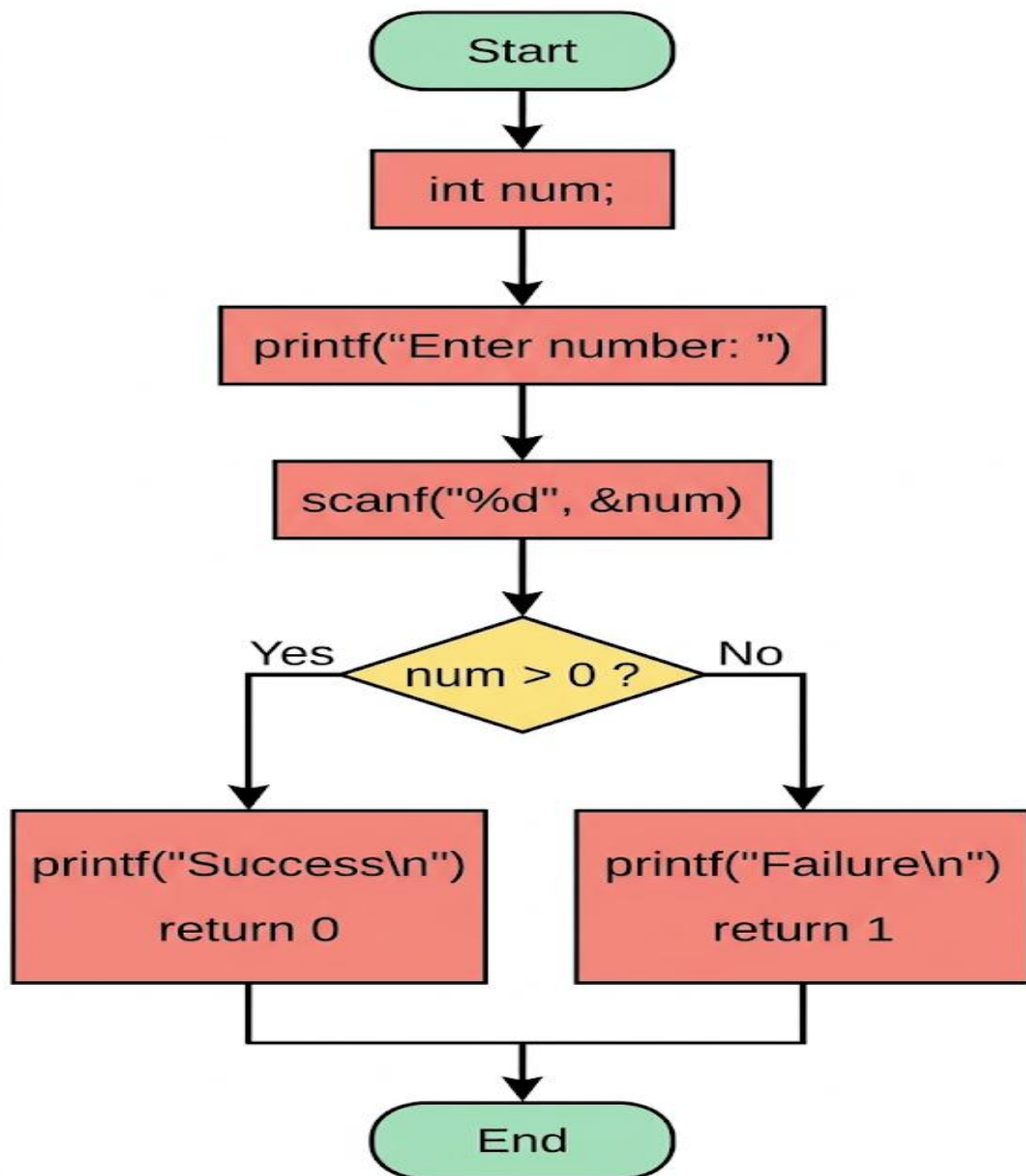


Figure 9 : Flow diagram of `task3_exit.c`

V. Task 4: Standard I/O Streams

OS Insight: The OS provides standard input (stdin) and standard output (stdout) streams. `scanf` reads from stdin, and `printf` writes to stdout.

Theory: The operating system provides two standard I/O streams for every process: standard input (stdin) and standard output (stdout). In C, the `scanf()` function reads data from stdin (typically the keyboard), while `printf()` writes data to stdout (typically the terminal screen). These streams allow a program to interact with the user through the shell without needing to know the specific hardware details behind them — the OS handles that abstraction.

A. C Source Code



```
GNU nano 8.7.1 task4_input.c
#include <stdio.h>

int main() {
    char name[50];

    printf("Enter your name: ");
    // Read string from standard input
    scanf("%s", name);

    printf("Hello, %s! Welcome to OS Class.\n", name);
    return 0;
}
```

Figure 10 : Source Code

B. Terminal Execution and Verification

```
pravina@LAPTOP-6K8UDRA5:~/lab3$ nano task4_input.c
pravina@LAPTOP-6K8UDRA5:~/lab3$ nano task4_input.c
pravina@LAPTOP-6K8UDRA5:~/lab3$ gcc task4_input.c -o task4
pravina@LAPTOP-6K8UDRA5:~/lab3$ ./task4
Enter your name: pravina
Hello, pravina! Welcome to OS Class.
pravina@LAPTOP-6K8UDRA5:~/lab3$ █
```

Figure 11 : Output

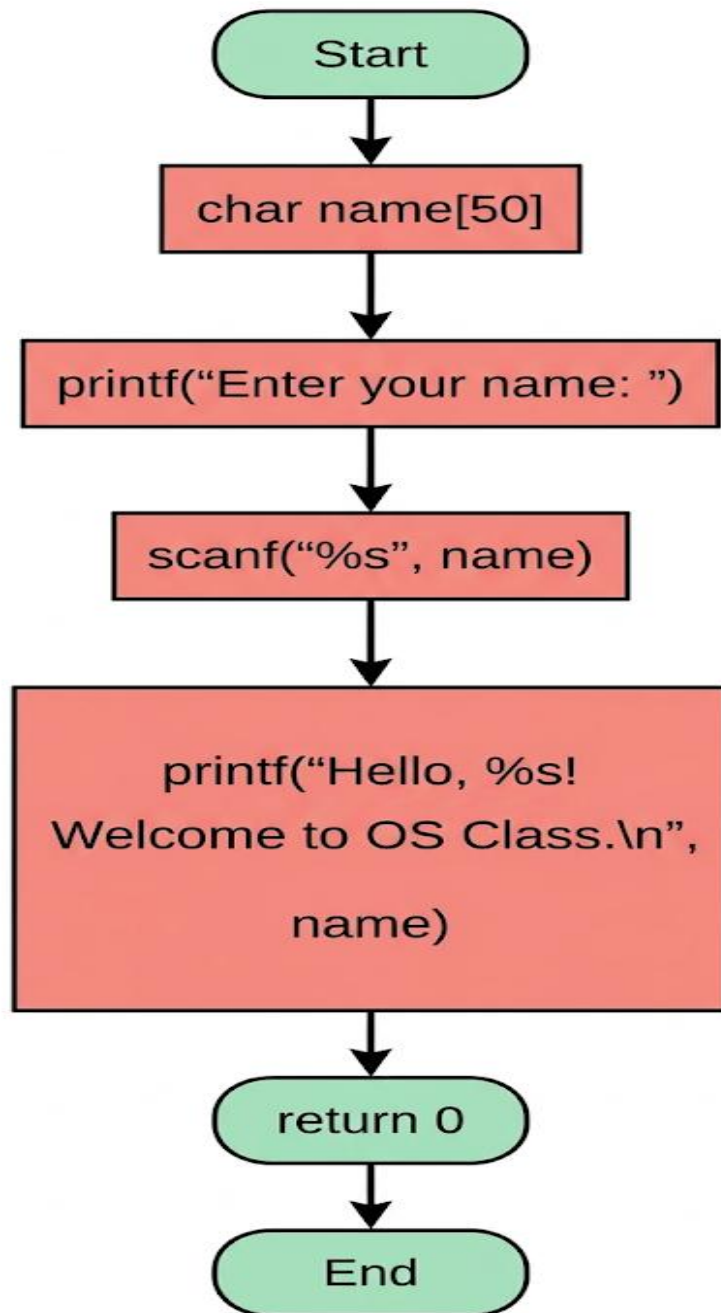


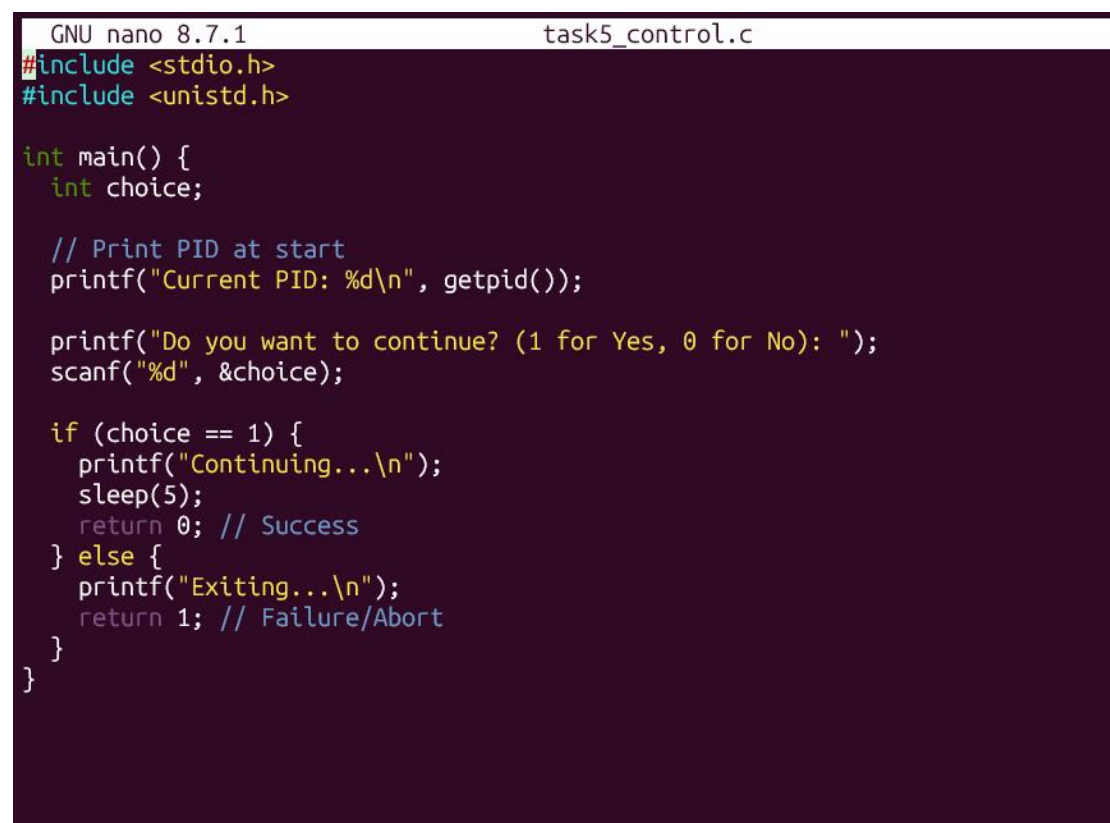
Figure 4 : Flow diagram of task4_input.c

VI. Task 5: Conditional Execution and Termination

OS Insight: Programs often branch based on user input. The OS tracks the final exit code to determine if the script or automation pipeline should continue or halt.

Theory: Programs often need to make decisions based on user input and communicate the outcome back to the OS. Using conditional statements (if-else), a program can branch its execution path depending on what the user enters. Combined with `getpid()` to identify the process and `sleep()` to simulate work, the program can then terminate with different exit codes - 0 for a normal/successful continuation, or 1 to signal an early exit or abort. This mirrors how real automation scripts and pipelines decide whether to proceed or halt based on a process's outcome.

A. C Source Code



```
GNU nano 8.7.1 task5_control.c
#include <stdio.h>
#include <unistd.h>

int main() {
    int choice;

    // Print PID at start
    printf("Current PID: %d\n", getpid());

    printf("Do you want to continue? (1 for Yes, 0 for No): ");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("Continuing...\n");
        sleep(5);
        return 0; // Success
    } else {
        printf("Exiting...\n");
        return 1; // Failure/Abort
    }
}
```

Figure 13 : Source Code

B. Terminal Execution and Verification

```
pravina@LAPTOP-6K8UDRA5:~/lab3$ nano task5_control.c
pravina@LAPTOP-6K8UDRA5:~/lab3$ ^C
pravina@LAPTOP-6K8UDRA5:~/lab3$ nano task5_control.c
pravina@LAPTOP-6K8UDRA5:~/lab3$ gcc task5_control.c -o task5
pravina@LAPTOP-6K8UDRA5:~/lab3$ ./task5
Current PID: 2452
Do you want to continue? (1 for Yes, 0 for No): 1
Continuing...
pravina@LAPTOP-6K8UDRA5:~/lab3$ ./task5
Current PID: 2462
Do you want to continue? (1 for Yes, 0 for No): 1
Continuing...
pravina@LAPTOP-6K8UDRA5:~/lab3$ echo $?
0
pravina@LAPTOP-6K8UDRA5:~/lab3$ ./task5
Current PID: 2481
Do you want to continue? (1 for Yes, 0 for No): 0
Exiting...
pravina@LAPTOP-6K8UDRA5:~/lab3$ echo $?
1
pravina@LAPTOP-6K8UDRA5:~/lab3$ █
```

Figure 14 : Output

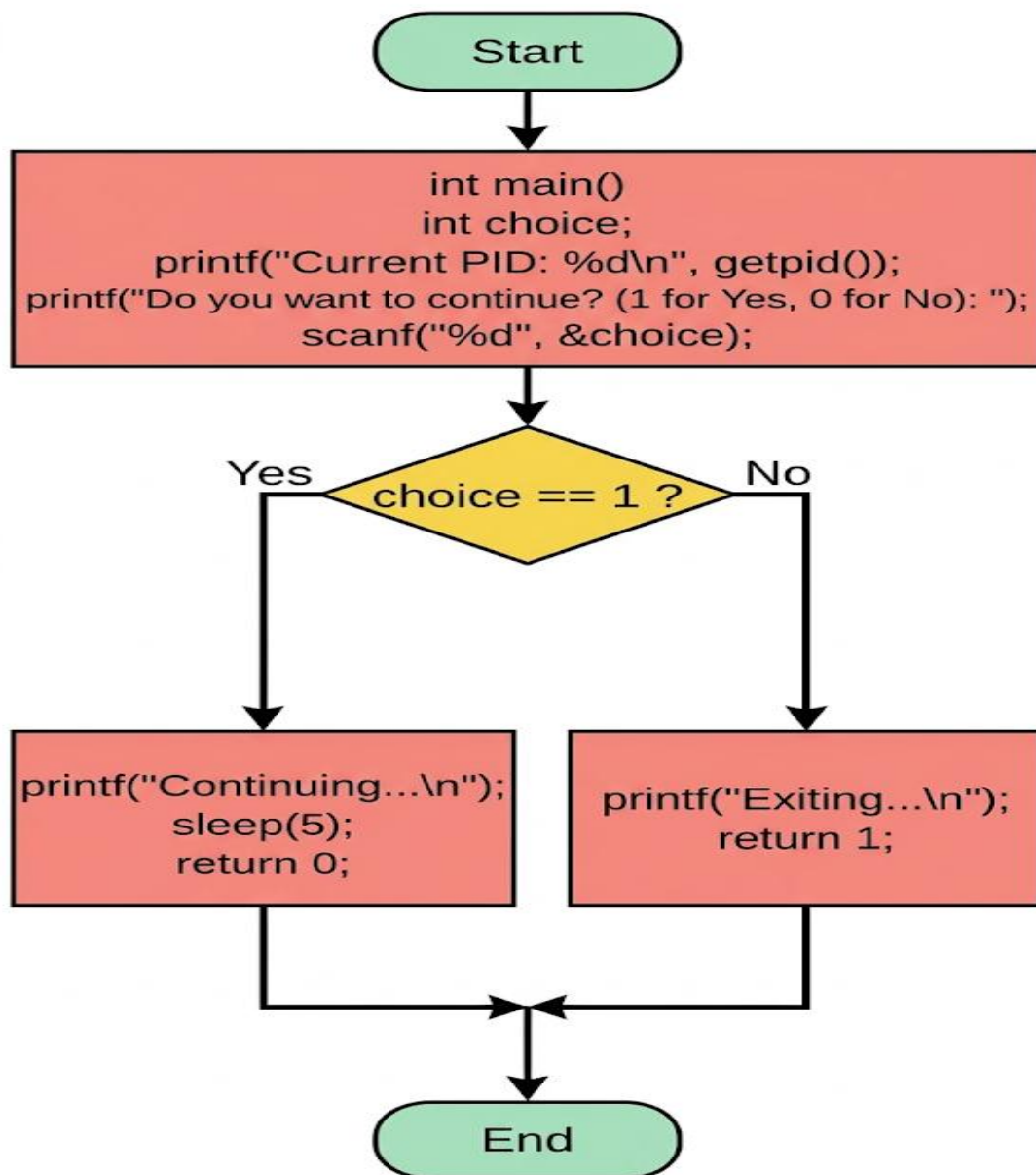


Figure 15 : Flow diagram of `task5_control.c`

VII. Conclusion

This lab demonstrated that C programs do not exist in a vacuum; they are active processes managed by the Linux kernel. By utilizing system calls like `getpid()`, `sleep()`, and return codes, we successfully bridged the gap between high-level C code and low-level OS process management.

Through the five tasks, we observed how a process moves through different states (running, sleeping, terminated), how the kernel assigns and tracks identity through PID and PPID, how standard I/O streams allow user interaction, and how exit codes let a program report success or failure back to the shell — feedback that is essential for scripting, automation, and process control in real enterprise systems.

GITHUB REPOSITORY

<https://github.com/Prabina-1/os-labs>

Appendix

```
pravina@LAPTOP-6K8UDRA5:~/os-labs$ git push origin main
Username for 'https://github.com': prabina-1
Password for 'https://prabina-1@github.com':
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 12 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (8/8), 1.40 KiB | 719.00 KiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Prabina-1/os-labs.git
   b945645..fd086bb  main -> main
```